Part A

Gemini Prompt: Using the text file attached can you answer these (1) compute column means, a correlation, and a simple lm() linear fit in R; and (2) do the same computation in Python (numpy/statsmodels).

Gemini Response:

1. R Code Implementation

```
# 1. Load the data (assuming the file is named 'roe01.txt')
data <- read.table("roe01.txt", header = TRUE)

# 2. Compute column means
col_means <- colMeans(data)
print("Column Means:")
print(col_means)

# 3. Compute correlation matrix (or correlation between ROE and ROEt)
# Full correlation matrix:
cor_matrix <- cor(data)
print("Correlation Matrix (Snippet):")
print(cor_matrix[1:5, 1:5])

# Specific correlation example (e.g., between ROE and ROEt)
specific_cor <- cor(data$ROE, data$ROEt)
cat("\nCorrelation between ROE and ROEt:", specific_cor, "\n")

# 4. Simple Linear Regression fit (e.g., predicting ROE using ROEt)
model <- lm(ROE ~ ROEt, data = data)
print("\nLinear Model Summary:")
summary(model)
```

Code Outputs:

"Column Means:"

ROEt	ATO	PM	LEV	GROWTH	PB	ARR	INV	ASSET	ROE
0.067778	0.429818	0.210952	0.708852	0.331254	2.126802	0.200780	0.100044	21.066002	0.410498

"Correlation Matrix (Snippet):"

	ROEt	ATO	PM	LEV	GROWTH
ROEt	1.00000000	0.03500163	0.10951819	-0.24454989	0.00954960
ATO	0.03500163	1.00000000	-0.27192103	-0.06517286	0.01858340
PM	0.10951819	-0.27192103	1.00000000	-0.12233415	0.02115707
LEV	-0.24454989	-0.06517286	-0.12233415	1.00000000	-0.01400840

```
GROWTH  0.00954960  0.01858340  0.02115707 -0.01400840  1.00000000
Correlation between ROE and ROEt: 0.5151322

"\nLinear Model Summary:"
Call:
lm(formula = ROE ~ ROEt, data = data)

Residuals:
    Min      1Q  Median      3Q     Max
-1.1366 -0.2984  0.0134  0.2571  5.5736

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.37384    0.02108   17.73   <2e-16 ***
ROEt         0.54080    0.04032   13.41   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.4674 on 498 degrees of freedom
Multiple R-squared:  0.2654,  Adjusted R-squared:  0.2639
F-statistic: 179.9 on 1 and 498 DF,  p-value: < 2.2e-16
```

Gemini Response:
2. Python Code Implementation (numpy and statsmodels)

```
import numpy as np
import statsmodels.api as sm

# 1. Load the data using numpy (skipping the header row)
# 'names=True' automatically treats the first row as column names
data = np.genfromtxt("roe01.txt", names=True)
column_names = data.dtype.names

# 2. Compute column means
print("Column Means:")
for col in column_names:
    mean_val = np.mean(data[col])
    print(f"{col}: {mean_val:.4f}")

# 3. Compute correlation
# Convert structured array to standard 2D numpy array to get a correlation matrix
data_2d = np.array([data[col] for col in column_names])
cor_matrix = np.corrcoef(data_2d)
```

```
print("\nCorrelation Matrix (first 3 columns):")
print(cor_matrix[:3, :3])

# Specific correlation example (e.g., ROE and ROEt index)
roe_idx = column_names.index("ROE")
roet_idx = column_names.index("ROEt")
specific_cor = cor_matrix[roe_idx, roet_idx]
print(f"\nCorrelation between ROE and ROEt: {specific_cor:.4f}")

# 4. Simple Linear Regression fit using statsmodels (Predict ROE using ROEt)
y = data["ROE"]
X = data["ROEt"]

# statsmodels requires manually adding a constant for the intercept
X = sm.add_constant(X)

# Fit the Ordinary Least Squares (OLS) model
model = sm.OLS(y, X).fit()

print("\nLinear Model Summary:")
print(model.summary())
```

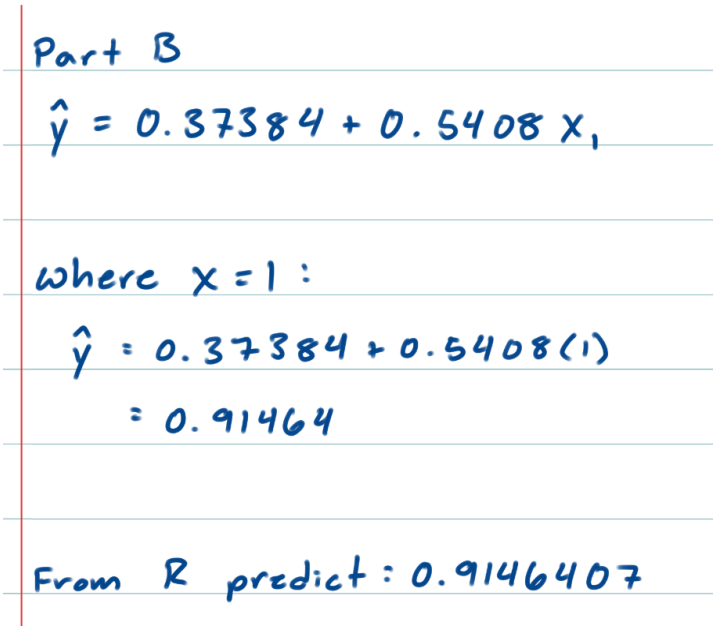
Code Outputs:

```
Column Means:
ROEt: 0.0678
ATO: 0.4298
PM: 0.2110
LEV: 0.7089
GROWTH: 0.3313
PB: 2.1268
ARR: 0.2008
INV: 0.1000
ASSET: 21.0660
ROE: 0.4105

Correlation Matrix (first 3 columns):
[[ 1.          0.03500163  0.10951819]
 [ 0.03500163  1.         -0.27192103]
 [ 0.10951819 -0.27192103  1.        ]]

Correlation between ROE and ROEt: 0.5151

Linear Model Summary:
                            OLS Regression Results
```

```
==============================================================================
Dep. Variable:                      y   R-squared:                       0.265
Model:                            OLS   Adj. R-squared:                  0.264
Method:                 Least Squares   F-statistic:                     179.9
Date:                Wed, 01 Jul 2026   Prob (F-statistic):           3.10e-35
Time:                        19:56:13   Log-Likelihood:                -328.19
No. Observations:                 500   AIC:                             660.4
Df Residuals:                     498   BIC:                             668.8
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.3738      0.021     17.734      0.000       0.332       0.415
x1             0.5408      0.040     13.412      0.000       0.462       0.620
==============================================================================
Omnibus:                      440.171   Durbin-Watson:                   1.926
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            32858.985
Skew:                           3.346   Prob(JB):                         0.00
Kurtosis:                      42.146   Cond. No.                         1.94
==============================================================================
```

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Part B



Part C

The number is wrong because the AI didn't calculate the average stock return, but rather the average of the underlying factor levels that R assigned to the data.

Fixed:

```
returns <- read.csv("returns.csv")
returns$date <- as.factor(returns$date)
avg <- mean(as.numeric(as.character(returns$stockA)), na.rm = TRUE)
```

Part D

While the outputs of the R and Python were the same, other than rounding differences, there could be differences that came from handling missing values and degrees of freedom in statistical calculations. R will return NA for missing values and will not skip them unless specifically told to, but Python library pandas will automatically skip the missing values and numpy will return them as nan. R will also use degree of freedom n-1 as the denominators for standard deviations and covariances, while numpy uses n as the denomination unless changing the ddof parameter.

Git link